19 · 全链路性能架构与技术栈选型

项	内容
文档编号	DT-PdM-19
依据	实测基准（2026-10-21）；ADR-013 修订
目标	诊断接口 P95 ≤ 200 ms（缓存命中）、≤ 800 ms（缓存穿透），单实例 ≥ 3000 req/s

---

1. 技术栈替换

1.1 Python 服务层

维度	原方案	新方案	理由
Web 框架	FastAPI（Starlette）	Litestar 2.25	路由/DI/序列化链路更薄；原生 msgspec；OpenAPI 不缺
ASGI 服务器	uvicorn（h11）	Granian 2.x（Rust/Hyper）	HTTP 解析与连接管理下沉到 Rust，同 worker 数吞吐更高
数据校验	pydantic v2	msgspec 0.22	Rust 侧编解码，零 `__dict__`，内存与延迟双降
CPU 密集调度	同步阻塞	`asyncio.to_thread` + 线程池	numpy 在 C 层释放 GIL，避免阻塞事件循环
缓存	无	L1 进程内 TTL + L2 Redis	见 §2

> 保留 `uvicorn` 作为本地对照与降级启动方式（同一 app 可被两者加载），不作为生产默认。

1.2 实测基准（同一 Litestar 应用，仅换服务器）

环境：Windows 11 开发机，2 workers，1500 请求，并发 40，请求体为无波形指标（命中缓存）。

服务器	吞吐	P50	P95	错误
Granian	717 req/s	37.1 ms	158.1 ms	0
uvicorn	590 req/s	42.8 ms	185.3 ms	0
差异	+21.5%	−13%	−14.7%	—

缓存统计：`hit_rate = 0.9994`（1549 命中 / 1 未命中），L2 未启用（测试环境无 Redis，自动降级）。

诚实说明：

- 这是开发机 + 缓存命中路径，不代表生产数字。Linux + 更多 worker + 真实波形解析会拉开更大差距（HTTP 层占比更高时 Granian 优势更明显）。
- 缓存命中率 99.94% 是压测特征（同一请求），真实场景预计 60% – 80%（多设备、多时间窗）。
- 生产验收指标以 `docs/08` 的性能测试为准，本表只用于证明"选型方向正确"。

---

2. 多级缓存设计

2.1 三层结构

```
■■
 ■
■■■ L1 ■■■■TTLCache■16 ■■■8192 ■■TTL 30s■
■     ■■■~1-5 µs■■■■
 ■
■■■ L2 Redis■TTL 120s■socket ■■ 0.2s■
■     ■■■~0.3-1 ms■■■ L1
 ■
■■■ ■■■numpy/scipy■5-50 ms■
```

2.2 关键设计

设计	取值	原因
分片	16 片	避免单字典锁竞争；单 key 命中不受全局影响
TTL（L1）	30 s	健康状态是时效数据，宁可多算不可给旧结论
TTL（L2）	120 s	跨实例共享，容忍度略高
容量	8192	约 20 基地 × 400 设备的热点集
Key	请求语义指纹（SHA-256 前 32 位）	剔除 trace_id/时间戳，同语义必命中
降级结果不缓存	`cacheable = not degraded`	否则数据恢复后仍返回"数据不足"，属于缓存放大故障
L2 故障	捕获所有异常，计 `l2_errors`，继续走 L1	缓存不得影响主流程

2.3 缓存键的构造

```
{
  "tenant": "site-a", "asset": "KA-101",
  "metrics": {"vib": 4.0, "base": 2.0, "k": 6.5, "t": 68.0, "tb": 55.0, "c": 3.0, "tr": 1.2},
  "wave": null,          # ■■■■■■■ fs/rpm/■■■/■■■■■■■■■■■■
  "model": "rules-v0.2.0"
}
```

为什么不对波形样本做哈希：一次 25600 点哈希约 0.3 ms，比计算本身还贵；且同一设备的相邻波形本就不同，命中率极低。用"几何 + 采样参数"做粗粒度键，配合短 TTL 即可。

2.4 缓存穿透与雪崩防护

风险	措施
穿透（大量不存在的 key）	计算前先校验租户与设备存在性（后续接资产服务）
雪崩（同时过期）	TTL 加 ±10% 抖动（待实现，见 §5）
击穿（热点 key 过期瞬间）	单实例内用 `asyncio.Lock` 做 key 级互斥（待实现）

---

3. 其他语言的高性能选型

语言	原计划	最终选型	理由
Rust	axum（未定）	axum + tokio + tower	生态最成熟；tower 中间件可复用限流/超时/重试；与 tonic（gRPC）同源

语言	原计划	最终选型	理由
Rust 序列化	serde_json	simd-json（热路径）	波形与特征批量解析，SIMD 加速 2 – 4 倍
Go	标准库 net/http	chi（路由）+ 标准库 http；高吞吐场景用 fasthttp	chi 零分配路由且兼容 `net/http`；fasthttp 仅在纯转发场景用（不兼容标准接口）
Go GraphQL	手写	gqlgen（代码生成）	编译期类型安全，无反射开销；支持 DataLoader 防 N+1
Go 序列化	encoding/json	bytedance/sonic	兼容 `encoding/json` API，性能 2 – 3 倍
Java	Spring Boot	Spring Boot 3 + 虚拟线程（Loom）；网关类用 Vert.x	虚拟线程解决阻塞 I/O 的线程成本；Vert.x 用于事件总线消费的高并发场景
Java 序列化	Jackson	Jackson + afterburner 模块	减少反射开销
消息	Kafka	Pulsar（已存在）	多租户 namespace + 分层存储

---

4. 全链路延迟预算

段	目标 P95	手段
边缘采集 → 特征	≤ 200 ms	Rust DSP，单线程批处理，零拷贝
特征 → 事件总线	≤ 50 ms	Pulsar 生产者批量 + 异步确认
事件 → 诊断服务	≤ 100 ms	消费者预取 + 常驻连接
诊断计算（缓存穿透）	≤ 500 ms	numpy 向量化；包络谱限制在 1 – 10 kHz 带
诊断计算（缓存命中）	≤ 5 ms	L1
诊断 → 读模型	≤ 3 s（最终一致）	投影器批量写
GraphQL 首屏	≤ 800 ms	DataLoader 批量 + 字段级懒加载
告警送达	≤ 60 s	直连推送，不走轮询

---

5. 待落地的性能项（Ph1 – Ph2）

#	项	预期收益	优先级
P1	TTL 抖动 + key 级互斥锁	消除雪崩/击穿	高
P2	波形计算走 Rust DSP 服务（gRPC）	包络谱从 40 ms → 8 ms	高
P3	特征值批量写入 TDengine（批 500 点）	写吞吐 ×5	高
P4	GraphQL DataLoader	首屏 6 次往返 → 1 次	高
P5	HTTP/2 + keep-alive + gzip/br	弱网首屏 − 30%	中
P6	Pulsar 消费者批量确认	消费吞吐 ×3	中

#	项	预期收益	优先级
P7	Redis 集群模式（当前单实例）	消除单点	中
P8	对象存储直传（预签名 URL）	波形上传不占应用带宽	低

---

6. 性能纪律（写进 CI）

- 任何 PR 若使 `tools/bench.py` 的 P95 劣化 > 20%，阻断合并。
- 新增依赖必须说明"为什么不能用现有组件"，避免技术栈膨胀。
- 缓存键变更必须同步更新 `cache_key_payload` 的版本号，避免新旧结果混用。
- 禁止在请求路径上做全表扫描、禁止在 JSON 字段上过滤（TiDB 无 GIN）。